Author Contributions Checklist Form

This form documents the artifacts associated with the article (i.e., the data and code supporting the computational findings) and describes how to reproduce the findings.

Part 1: Data

☐ This paper **does not** involve analysis of external data (i.e., no data are used or the only data are generated by the authors via simulation in their code).

☒ I certify that the author(s) of the manuscript have legitimate access to and permission to use the data used in this manuscript.

Abstract

We use a crop-yield dataset from the Kellogg Biological Station Long-Term Ecological Research project. The publicly available data spans from 1989-2023 and comprises three crops (Zea mays L., Triticum aestivum L., and Glycine max L.) and four treatments, descriptions of which can be found here: https://lter.kbs.msu.edu/research/long-term-experiments/main-cropping-system-experiment/. The response we consider is kilograms of yield per hectare of area of crop. For the most part, there is one crop harvested per year along with the data containing 6 replicates of yield/area values across each of the 4 treatments (though there are some exceptions, which are described in the supplementary materials).

Availability

☒ Data **are** publicly available
☐ Data **cannot be made** publicly available

If the data are publicly available, see the *Publicly available data* section. Otherwise, see the *Non-publicly available data* section, below.

Publicly available data

☒ Data are available online at: https://lter.kbs.msu.edu/datatables/51
☒ Data are available as part of the paper's supplementary material.
☐ Data are publicly available by request, following the process described here:

☐ Data are or will be made available through some other mechanism, described here:

Non-publicly available data

Discussion of lack of publicly available data:

Description

File format(s)

☒ CSV or other plain text: experiment/cleaned_agronomic_data.csv
☐ Software-specific binary format (.Rda, Python pickle, etc.):
☐ Standardized binary format (e.g., netCDF, HDF5, etc.):
☐ Other (described here):

Data dictionary

☒ Provided by the authors in the following file(s): experiment/agronomic_data.csv
☐ Data file(s) is (are) self-describiing (e.g., netCDF files)
☐ Available at the following URL:

Additional information (optional)

The file experiment/agronomic_data.csv is the original data csv, which contains experiment notes/comments. The file experiment/cleaned_agronomic_data.csv has these comments removed so that the data is easily readable using Pandas.

Part 2: Code

Abstract

Our code contains (setting/experiment-specific) implementations of the following: weighted MC randomization test, unweighted MCMC randomization test, resampling algorithms, and adaptive assignment algorithms. It contains all necessary python scripts to replicate our analyses, Jupyter notebooks to produce plots, and bash files to run code on a cluster.

Description

Code format(s)

☒ Script files
- ☐ R ☒ Python ☐ Matlab
- ☐ Other:

☐ Package
- ☐ R ☐ Python ☐ MATLAB toolbox
- ☐ Other:

☐ Reproducible report
- ☐ R Markdown ☐ Jupyter notebook
- ☐ Other:

☒ Shell script

☒ Other (described here):

Markdown (README file)

Supporting software requirements

Version of primary software used

Python version 3.10.14

Libraries and dependencies used by the code

Numpy version 1.26.4

Sklearn version 1.4.1.post1
Pandas version 2.2.3
Matplotlib version 3.9.4

Supporting system/hardware requirements (optional)

Access to CPU cluster

Parallelization used

☐ No parallel code used
☐ Multi-core parallelization on a single machine/node
Number of cores used:
☒ Multi-machine/multi-node parallelization
Number of nodes and cores used: ~500 total cores (roughly 10 nodes and 50 cores each)

License

☒ MIT License (default)
☐ BSD
☐ GPL v3.0
☐ Creative Commons
☐ Other (described here):

Additional information (optional)

Part 3: Reproducibility workflow

Scope

The provided workflow reproduces:

☒ Any numbers provided in text in the paper

☒ The computational method(s) presented in the paper (i.e., code is provided that implements the method(s))

☒ All tables and figures in the paper

☐ Selected tables and figures in the paper, as explained and justified here:

Workflow details

Location

The workflow is available:

☒ As part of the paper's supplementary material

☒ In this Git repository: https://anonymous.4open.science/r/RTs-for-AdaptiveData-530F/

☐ Other:

Format(s)

☐ Single master code file

☒ Wrapper (shell) script(s)

☒ Self-contained R Markdown file, Jupyter notebook, or other literate programming approach

☒ Text file (e.g., a readme-style file) that documents workflow

☐ Makefile

☐ Other (more detail in 'Instructions' below)

Instructions

The bash scripts run_all_for_setting.sh and run_all_varying_m_for_setting.sh, each take a single command-line argument denoting the simulation setting. Running both of these scripts at any given setting input produces all simulation results for that setting, where the setting inputs

are 0–6 and correspond, respectively, to the settings described in: Section 4.2.1, Section 4.1.2, Section 4.1.1, Appendix N.2.3, Section 4.2.2, Appendix N.1.4, Appendix N.1.3. The script run_all_varying_dim.sh takes no inputs and reproduces the varying dimension results in Appendix N.1.3. Together, these three scripts reproduce all hypothesis testing simulation results in our paper.

The script run_all_interval.sh produces the confidence interval and conformal prediction interval (with no sample sharing) results and takes in a single command line argument---either 'confidence' or 'conformal'. Finally, the script run_all_share_conformal.sh reproduces all simulation results involving conformal prediction interval construction using sample sharing. The python scripts collate_results.py and collate_interval_results.py collate all results and should be called only after the jobs submitted in the aforementioned bash scripts have finished running. The script collate_results.py takes in the same setting command-line argument and collates results for that setting while collate_interval_results.py takes the single command-line argument, 'confidence' or 'conformal', and collates all corresponding interval results (including those using sample sharing in the case of conformal). Both the bash scripts and the python scripts for collation should be run in the current directory on a compute cluster. Once this is done, the directories collated_results, and collated_interval_results should be transferred to a local machine in the same directory as the Jupyter notebook visualizer.ipynb, whereupon this notebook can be executed to reproduce all of our plots.

For the real data analysis, navigate to the experiment directory and submit the SLURM scripts stationarity_runscript.sh and cond_indep_runscript.sh. Once they have finished running, the files collate_stationarity.py and collate_cond_indep.py should be run to collate the results to produce LaTeX tables of results, stored in the directory experiment/latex_tables.

Expected run-time

Approximate time needed to reproduce the analyses on a standard desktop machine:

☐ <1 minute

☐ 1-10 minutes

☐ 10-60 minutes

☐ 1-8 hours

☐ >8 hours

☒ Not feasible to run on a desktop machine, as described here:

Our code is set up to be parallelized across multiple machines/cores on a CPU cluster.

Additional documentation (optional)

Please see me the README.md file in the supplementary materials.

Notes (optional)